

WHAT THIS SESSION IS

Run on 20 September 2026, and it closes the MCP block. Akshay Agarwal starts by answering the two questions left hanging yesterday, then builds four more demos: a tool on one server calling a tool on another, error handling that keeps server internals off the client, a full authentication and authorization server with tokens and scopes, and finally a client for GitHub's own MCP server that creates a repository, commits a file, opens a pull request and comments on it. There is almost no LLM in this session, and that is deliberate. Once you can call tools directly, the protocol is the only thing left to learn.

CONTENTS

1. Answers to yesterday's open questions
2. Why one tool would call another
3. A server that is also a client
4. Calling into the other direction
5. The client, and three ways to call a tool
6. Two kinds of errors
7. ToolError and masking
8. Seeing errors from the client side
9. Log levels
10. Authentication: the four steps
11. The secure server: keys, clients and scopes
12. Protecting tools with scopes
13. The token route
14. The client: getting a token and using it
15. Five scenarios, and what each proves
16. Token expiry
17. Using somebody else's server: GitHub
18. Creating the token on GitHub
19. The GitHub client
20. The full pull request flow
21. What went wrong live
22. Clarifications from the Q&A

1 Answers to yesterday's open questions

Three things were left open at the end of session 9, and the class opened with them.

How do you know which library versions go together?

The honest answer given in class: nobody remembers version compatibility. Copy the `dependencies` block out of `pyproject.toml`, paste it into an assistant such as ChatGPT or Claude, ask it to fix the versions, put the result back and run `uv sync`. Before AI tools this was a Stack Overflow job. The instructor also pointed to the course Discord: paste the file there and someone from the team can look.

Can we see the raw JSON-RPC messages?

Not yet. Code was tried before class to print the actual handshake and tool call messages, but it was not considered good enough to teach from, so it was postponed rather than shown half finished. It is still on the list, probably for the agents sessions.

Can a tool call another tool?

Yesterday this was answered only for two functions in the same file. The real question is a tool on one server calling a tool on a different server, and that is the first demo today.

A DETAIL FROM THE NOTIFICATION RECAP

The callbacks demo from yesterday was walked through again, and the handler `on_log` was renamed to `test` to make a point: these are your own functions with your own names. What matters is which keyword you pass them to, `on_logging_message` or `on_progress`. The shared code for today keeps that rename, so do not be confused when you open it.

2 Why one tool would call another

Before the code, the reason. You will often want your own MCP server in front of somebody else's. The example from class: your agent needs GitHub, but you do not want the raw GitHub response and you want your own checks to run first. So you write your own server with your own tool, and inside that tool you call the GitHub server.

```

your client
  |
  v
+-----+          +-----+
| YOUR MCP SERVER |    | GITHUB MCP SERVER |
|                 |    |                   |
| tool: open_pr()  | ----> | tool: create_pull_... |
|   check the branch   |    |                   |
|   call GitHub       |    +-----+
|   clean up the answer |
+-----+

```

the server is a server to your client, and a client to GitHub

All four combinations work: stdio calling stdio, stdio calling HTTP, HTTP calling stdio, and HTTP calling HTTP. The demo shows two of them in opposite directions.

3 A server that is also a client

05_tool_to_tool/stdio_server.py

```
from fastmcp import Client, FastMCP

mcp = FastMCP("Stdio Server")

@mcp.tool
def double(n: int) -> int:
    """Return n multiplied by 2."""
    return n * 2

@mcp.tool
def add_then_double(a: int, b: int) -> int:
    """Add two numbers, then double the result."""
    return double(a + b)

@mcp.tool
async def sum_of_squares(a: int, b: int) -> int:
    """Return a*a + b*b, using the square tool of the HTTP server."""
    async with Client("http://127.0.0.1:8002/mcp") as http:
        square_a = await http.call_tool("square", {"n": a})
        square_b = await http.call_tool("square", {"n": b})
    return square_a.data + square_b.data

if __name__ == "__main__":
    mcp.run(transport="stdio", show_banner=False)
```

Two different things are happening here, and it is worth separating them.

- `add_then_double` calls `double` as a plain Python function. Same file, same process, no protocol involved. This is what was shown yesterday.
- `sum_of_squares` is the new thing. `Client` comes from FastMCP itself. Give it a URL and you have an MCP client inside your server. `call_tool(name, args)` runs a tool on the other server, and `.data` is the value that comes back.
- `async with` opens the connection and closes it when the block ends, which is the same session idea from session 9.
- The tool becomes `async`, because talking to another server takes time and should not block.

4 Calling into the other direction

05_tool_to_tool/http_server.py

```
import sys
from pathlib import Path

from fastmcp import Client, FastMCP
from fastmcp.client.transports import PythonStdioTransport

mcp = FastMCP("HTTP Server")

@mcp.tool
def square(n: int) -> int:
    """Return n multiplied by itself."""
    return n * n

@mcp.tool
async def double_then_square(n: int) -> int:
    """Double n, then square it, using the double tool of the stdio server."""
    stdio_server = PythonStdioTransport(
        Path(__file__).parent / "stdio_server.py", python_cmd=sys.executable)
    async with Client(stdio_server) as stdio:
        doubled = await stdio.call_tool("double", {"n": n})
    return square(doubled.data)

if __name__ == "__main__":
    mcp.run(transport="http", host="127.0.0.1", port=8002)
```

Same idea, other transport. A URL is enough for an HTTP server, but for a stdio server the client has to start a process, so you hand `Client` a `PythonStdioTransport` with the file to run and the interpreter to run it with. That is the same pair of values you have been putting in `command` and `args` all along.

sum_of_squares (on stdio server)

`Client("http://127.0.0.1:8002/mcp")`

|

v

HTTP server's square

|

9 + 16 = 25

double_then_square (on HTTP server)

`PythonStdioTransport("stdio_server.py")`

|

v

stdio server's double

|

3 -> 6 -> 36

5 The client, and three ways to call a tool

05_tool_to_tool/client.py

```
import asyncio
import sys
from pathlib import Path

from fastmcp import Client
from fastmcp.client.transports import PythonStdioTransport

stdio_server = PythonStdioTransport(
    Path(__file__).parent / "stdio_server.py", python_cmd=sys.executable)
http_server = "http://127.0.0.1:8002/mcp"

async def main():
    async with Client(stdio_server) as stdio, Client(http_server) as http:

        result = await stdio.call_tool("add_then_double", {"a": 3, "b": 4})
        print(f'1. {result.data}')

        result = await stdio.call_tool("sum_of_squares", {"a": 3, "b": 4})
        print(f'2. {result.data}')

        result = await http.call_tool("double_then_square", {"n": 3})
        print(f'3. {result.data}')

if __name__ == "__main__":
    asyncio.run(main())
```

Start the HTTP server first, then run the client. The output, which we reproduced with the class code:

```
1. 14      # (3 + 4) * 2, all inside the stdio server
2. 25      # 3*3 + 4*4, the stdio server called the HTTP server
3. 36      # (3 * 2) squared, the HTTP server called the stdio server
```

This client uses FastMCP's own `Client` rather than `MultiServerMCPClient`. There is no LangChain and no model anywhere. A student asked the obvious question: without an LLM, how does the client know which tool to use? Because you wrote the name yourself. The model is only needed when you want something else to decide.

Way	How you call	How you read the result	Good for
LangChain tool from the adapter	<code>await tool.ainvoke(args)</code>	<code>result[0]['text']</code>	Giving tools to a model with <code>bind_tools</code>
Adapter session	<code>await session.call_tool(name, args)</code>	<code>result.content[0].text</code> , plus <code>result.isError</code>	Direct calls where you want the error flag
FastMCP <code>Client</code>	<code>await client.call_tool(name, args)</code>	<code>result.data</code>	Server to server calls, quick scripts

THE FIVE MINUTES LOST TO A MISSING LINE

The first run printed nothing at all. No error, no output. The cause was that `asyncio.run(main())` had not been written, so `main()` was never called. It happened twice in this session, in this demo and again in the authentication client. If an async script produces silence, check the bottom of the file first.

6 Two kinds of errors

Folder 06 is about what the client should see when a tool fails. The split made in class is the one that matters:

	Expected	Unexpected
Example	Dividing by zero	A file the tool opens is missing
Who knows about it	You. You wrote the check	Nobody. It is a bug
What the client should see	A clear message it can act on	A generic message, nothing more
How	<code>raise ToolError("...")</code>	<code>mask_error_details=True</code>

7 ToolError and masking

06_error_handling/server.py

```
from pathlib import Path
from fastmcp import FastMCP
from fastmcp.exceptions import ToolError

# mask_error_details = for UNEXPECTED errors, the client only gets a
# generic message. The real error stays on the server.
mcp = FastMCP("StdioServer", mask_error_details=True)

@mcp.tool
def divide(num1: int, num2: int) -> float:
    """Divide num1 by num2."""
    if num2 == 0:
        raise ToolError("Cannot divide by zero. "
                        "Please use a different number for num2.")
    return num1 / num2

@mcp.tool
def read_secret_file() -> str:
    """A tool with a bug: it opens a file that does not exist."""
    return open(Path(__file__).parent / "passwords.txt").read()

if __name__ == "__main__":
    mcp.run(transport="stdio", show_banner=False)
```

- `ToolError` is FastMCP's way of saying "this message is meant for the caller". Its text is always sent through, even with masking on.
- `mask_error_details=True` covers everything else. An ordinary exception becomes a short generic message, and the real traceback stays in the server logs.
- `read_secret_file` is a deliberate bug. Nothing catches the missing file.

WHY MASKING MATTERS

Without it, the client gets the full Python error, including the absolute path of the file the server tried to open. That is your directory layout handed to a stranger. The class compared it to a web application: users get a 500, not a stack trace. The same habit applies here, and it matters more when the tool touches personal data.

8 Seeing errors from the client side

06_error_handling/client.py

```
async def try_tool(session, tool_name: str, arguments: dict):
    print(f"\nCalling {tool_name}({arguments})")

    result = await session.call_tool(tool_name, arguments)

    tool_output = result.content[0].text
    if result.isError:
        print(" ERROR ->", tool_output)
    else:
        print(" OK    ->", tool_output)
```

This is the shape to remember. A failed tool is not an exception on the client. It is a normal result with `isError` set to true, and the message sits in the same place the answer would have been.

The client connects with a session, as in session 9, and adds one environment setting:

```
"stdio_server": {
    "transport": "stdio",
    "command": sys.executable,
    "args": [str(SERVER_PATH)],
    # Hide the server's own error logs, so we only see what
    # the CLIENT receives.
    "env": {"FASTMCP_LOG_LEVEL": "CRITICAL"},
}
```

Without that line you see the server's traceback and the client's message mixed together, which hides the very thing the demo is about. Running all three calls gives this, which we reproduced with the class code:

```
Calling divide({'num1': 10, 'num2': 2})
OK    -> 5.0

Calling divide({'num1': 10, 'num2': 0})
ERROR -> Cannot divide by zero. Please use a different number for num2.

Calling read_secret_file({})
ERROR -> Error calling tool 'read_secret_file'
```

Set `mask_error_details=False` and the last line becomes the real error instead, naming the missing file and its full path. That comparison was run live, and it is worth running yourself once.

9 Log levels

The `FASTMCP_LOG_LEVEL` value follows the ordinary Python logging levels, which the class went over quickly because they are the same everywhere, from log4j to any web service:

```
DEBUG < INFO < WARNING < ERROR < CRITICAL
```

```
set it to DEBUG    -> you see everything
```

```
set it to CRITICAL -> you see almost nothing
```

```
the level you set is the LOWEST that gets through
```

10 Authentication: the four steps

The biggest topic of the session, and it was drawn before any code. MCP does not invent anything here. It is the same client credentials flow used by ordinary APIs.

CLIENT		SERVER
1. POST /token		
client_id + client_secret	----->	check them against the registered clients
2. access token (a Bearer token)	<-----	
3. tools/call with header		
Authorization: Bearer <token>	----->	verify the token, check the scopes
4. the tool result	<-----	

- **Client id and client secret** identify an application, not a person. That is the answer to the question about usernames and passwords in class: a human logs in with a username, an application authenticates with an id and a secret.
- **The access token** means the client does not send its secret again on every call.
- **Bearer** is the token type. Whoever carries it may use it, which is exactly why it expires.
- **Scopes** come along with the token and decide what the holder may do. Authentication is who you are, authorization is what you may do.

11 The secure server: keys, clients and scopes

07_authentication/server.py, the setup

```
import hmac # compare_digest = safe way to compare secrets

from fastmcp import FastMCP
from fastmcp.server.auth import require_scopes
from fastmcp.server.auth.providers.jwt import JWTVerifier, RSAKeyPair
from fastmcp.server.dependencies import get_access_token

HOST = "127.0.0.1"
PORT = 8004

ISSUER = f"http://{HOST}:{PORT}" # who created the token
AUDIENCE = "secure-mcp-server" # "aud": which server the token is for
TOKEN_LIFETIME_SECONDS = 3

key_pair = RSAKeyPair.generate() # public/private keys, access token

REGISTERED_CLIENTS = {
    "reader-app": {
        # client-id
        "secret": "reader-secret", # client-secret
        "allowed_scopes": ["app:read"],
    },
    "admin-app": {
        "secret": "admin-secret",
        "allowed_scopes": ["app:read", "db:reset"],
    },
}

mcp = FastMCP(
    "Production-style Secure Server",
    auth=JWTVerifier(
        public_key=key_pair.public_key,
        issuer=ISSUER,
        audience=AUDIENCE,
    )
)
```

- **JWT** is the token format. A JSON Web Token carries claims such as `sub` (who owns it), `iss` (who issued it), `aud` (which service it is for), the scopes, and `exp` (when it dies).
- **`RSAPair.generate()`** makes a private key to sign tokens and a public key to verify them. Printing them live showed the familiar key block, with the private key masked. In production you do not generate a fresh pair at startup, you load an existing one, and usually another service issues the tokens entirely.
- **`JWTVerifier`** is what turns the server into a protected one. It checks the signature with the public key, and rejects anything with the wrong issuer or audience.
- **`REGISTERED_CLIENTS`** is a dictionary only to keep the demo readable. The instructor was explicit that this belongs in a database in a real system.
- **Scope names** are yours to choose. The convention shown was `resource:action` : `app:read` , `db:reset` , or in a real product `courses:read` , `logs:write` , `employee:update` .
- **`TOKEN_LIFETIME_SECONDS = 3`** is absurdly short on purpose, so the expiry demo in section 16 works. Five minutes is a normal value.

12 Protecting tools with scopes

```
@mcp.tool(auth=require_scopes("app:read"))
def greet(name: str) -> str:
    """Return a friendly greeting message for the given person's name."""
    return f"Hello, {name}! Welcome to the MCP class."

@mcp.tool(auth=require_scopes("app:read"))
def whoami() -> str:
    """Tell the caller who the server thinks they are, and what they may do."""
    token = get_access_token()
    return f"You are '{token.client_id}' with scopes {token.scopes}"

@mcp.tool(auth=require_scopes("db:reset"))
def reset_database() -> str:
    """Reset the database. Dangerous - needs the db:reset scope!"""
    print("[server] reset_database was called!")
    return "Database reset done."
```

- **`auth=require_scopes(...)`** guards one tool. Pass several names and the caller needs *all* of them.
- **`get_access_token()`** hands you the verified token inside a tool, so you can read `client_id` and `scopes` . That is how `whoami` answers.
- **`reader-app`** can call `greet` and `whoami` . **`admin-app`** can call all three.

A DETAIL WORTH KNOWING: THE TOOL DISAPPEARS

When the caller lacks the scope, FastMCP does not answer "forbidden". It filters the tool out of `tools/list` altogether, and a direct call comes back as not found. In the live run the message was `Unknown tool: 'reset_database'`. We checked this in the FastMCP source to be sure, and the internal message is "not found or not authorized".

That is good security. A caller cannot even learn that a dangerous tool exists. It is also a debugging trap: if a tool you know you wrote comes back as unknown, check the scopes on your token before you go looking for typos.

The same source makes another thing explicit: on stdio there is no auth concept and this filtering is skipped. Authentication here is an HTTP idea, which is why this whole demo runs over HTTP.

13 The token route

The server also has to hand out tokens, and that is not an MCP tool. It is an ordinary HTTP endpoint sitting next to the MCP endpoint.

```

@mcp.custom_route("/token", methods=["POST"])
async def issue_token(request: Request) -> JSONResponse:
    form = await request.form()

    #1. Check
    if form.get("grant_type") != "client_credentials":
        return error_response("unsupported_grant_type", 400)

    #2. Check the client
    client_id = str(form.get("client_id", ""))
    client_secret = str(form.get("client_secret", ""))
    client = REGISTERED_CLIENTS.get(client_id)

    if not is_valid_client(client, client_secret):
        return error_response("invalid_client", 401)

    #3. Check the scope
    requested_scopes = (str(form.get("scope", "")).split()
                        or client["allowed_scopes"])
    for scope in requested_scopes:
        if scope not in client["allowed_scopes"]:
            return error_response("invalid_scope", 400)

    access_token = key_pair.create_token(
        subject=client_id,      # "sub": who owns the token
        issuer=ISSUER,          # this server is the issuer here
        audience=AUDIENCE,     # "aud": this MCP server
        scopes=requested_scopes, # what the token allows
        expires_in_seconds=TOKEN_LIFETIME_SECONDS, # "exp"
    )

    return JSONResponse({
        "access_token": access_token,
        "token_type": "Bearer",
        "expires_in": TOKEN_LIFETIME_SECONDS,
        "scope": " ".join(requested_scopes)
    })

```

- `@mcp.custom_route(...)` adds a plain web route to the same server. FastMCP documents it for health checks, status endpoints and webhooks, and a token endpoint fits the same shape.

- **Check 1** is the standard OAuth `grant_type` field. It is a formality, and the instructor said as much, but real clients send it so real servers check it.
- **Check 2** compares secrets with `hmac.compare_digest` rather than `==`. Both work, but the safe version always takes the same amount of time, so an attacker cannot learn the secret one character at a time by measuring how fast the answer comes back.
- **Check 3** refuses any scope the client is not registered for. If the client asks for nothing, it gets exactly its registered scopes.
- Only after all three does the server sign a token.
- The error helper returns `{"error": "..."} with an HTTP status code, which is the ordinary OAuth error shape.`

14 The client: getting a token and using it

```
import httpx # a simple HTTP library, used to call the /token endpoint

BASE_URL = "http://127.0.0.1:8004"
TOKEN_URL = f"{BASE_URL}/token"
MCP_URL = f"{BASE_URL}/mcp"

async def get_token(client_id: str, client_secret: str) -> str:
    form = {
        "grant_type": "client_credentials",
        "client_id": client_id,
        "client_secret": client_secret,
    }

    async with httpx.AsyncClient() as http:
        response = await http.post(TOKEN_URL, data=form)

    if response.status_code == 401:
        print("get_token -> REJECTED:", response.json()["error"])
        return None

    return response.json()["access_token"]
```

Getting the token is a plain HTTP POST, not MCP. Notice no scope is sent, so the server decides what this client may do. Using the token is where MCP comes back:

```

async def call_tool(token: str | None, tool_name: str, args: dict):
    connection = {
        "transport": "streamable_http",
        "url": MCP_URL
    }

    if token is not None:
        connection["headers"] = {"Authorization": f"Bearer {token}"}
    client = MultiServerMCPClient({"secure": connection})

    try:
        async with client.session("secure") as session:
            result = await session.call_tool(tool_name, args)
            status = "DENIED" if result.isError else "OK"
            print(f"{tool_name} -> {status}:", result.content[0].text)
    except Exception as error:
        # The MCP client wraps failures in an exception group; unwrap it if so.
        real_error = (error.exceptions[0]
                      if isinstance(error, BaseExceptionGroup) else error)
        print(f"{tool_name} -> REJECTED:", str(real_error).splitlines()[0])

```

- The only change to a normal connection is the `headers` entry. That single header is the whole of using a protected MCP server, and it will be the same for GitHub later.
- Two failure shapes have to be handled, and this is the part worth studying. A rejected *connection* raises an exception, because the server never lets you in. A rejected *tool* comes back as a normal result with `isError` true.
- `BaseExceptionGroup` appears because the MCP client runs things concurrently and wraps failures in a group. Unwrapping it gives the real message instead of a confusing one.

15 Five scenarios, and what each proves

```

async def main():
    #1. token = None
    await call_tool(token=None, tool_name="greet", args={"name": "Akshay"})

    #2. Reader Token
    reader_token = await get_token(client_id="reader-app",
                                    client_secret="reader-secret")
    await call_tool(token=reader_token, tool_name="greet",
                    args={"name": "Akshay"})

    time.sleep(5)

    #3. Out of Scope
    await call_tool(token=reader_token, tool_name="reset_database", args={})

    #4: Using Admin Access token
    admin_token = await get_token(client_id="admin-app",
                                   client_secret="admin-secret")
    await call_tool(token=admin_token, tool_name="reset_database", args={})

    #5. If the creds are wrong
    admin_token = await get_token(client_id="admin-app",
                                   client_secret="admin-secret1")

```

#	What is tried	What comes back	What it proves
1	No token at all	401 Unauthorized , raised before any tool runs	The server is actually protected
2	Reader token, greet	OK: Hello, Akshay! ...	Authentication works
3	Reader token, reset_database	DENIED: Unknown tool	Scopes are enforced, and the tool is hidden
4	Admin token, reset_database	OK: Database reset done.	The same call succeeds with the right scope
5	Wrong client secret	REJECTED: invalid_client	Bad credentials never get a token

CAREFUL WITH SCENARIO 3 IN THE SHARED CODE

In the file as shared there is a `time.sleep(5)` just above scenario 3, while the token lives for 3 seconds. So when you run it, scenario 3 fails with `401 Unauthorized` because the token has *expired*, not because of the scope. We ran it both ways to be sure. Remove that sleep, or set `TOKEN_LIFETIME_SECONDS` back to 300, and scenario 3 shows what it is meant to show:

```
with the sleep    reset_database -> REJECTED: Client error '401 Unauthorized'
without the sleep reset_database -> DENIED: Unknown tool: 'reset_database'
```

Both outcomes are correct and both are worth seeing. Just know which one you are looking at.

16 Token expiry

The expiry demo is the sleep above, done on purpose. Lifetime three seconds, wait five, call again, and the token that worked a moment ago is refused.

```
t=0s  get_token()      -> token valid until t=3s
t=0s  greet           -> OK
t=5s  reset_database   -> 401, the token is dead
t=5s  get_token() again -> a fresh token
t=5s  reset_database   -> OK
```

Nothing was revoked and nothing changed on the server. The `exp` claim inside the token simply passed. This is why real clients keep a refresh token or just ask for a new one when they get a 401, which is what a student asked about at the end and the answer was that MCP handles it exactly like any other API.

17 Using somebody else's server: GitHub

The last demo is the payoff of the whole MCP block. No server of our own this time. GitHub publishes an MCP server, so we only write a client.

	Value
Endpoint	<code>https://api.githubcopilot.com/mcp/</code>
Transport	Streamable HTTP
Auth	<code>Authorization: Bearer <your token></code>
Cost	Free to use with a GitHub account

Look at that header. It is the same one you built by hand in section 14. Everything you learned on your own server is what you now use against a real one.

18 Creating the token on GitHub

Done live in GitHub settings, under developer settings, personal access tokens. The choices made:

- **Fine-grained token**, not classic. Fine-grained tokens let you pick exact permissions and a specific set of repositories.
- **Short expiry**. One day, because it was needed for one class.
- **Permissions** for the demo: administration (to create a repository), contents (to add a file), pull requests (to open and comment), and actions. Each set to read and write, since the default is read only.

The token then goes in `.env`, never in the code:

```
GITHUB_PERSONAL_ACCESS_TOKEN=your-github-token-here
```

TREAT THIS TOKEN LIKE A PASSWORD

It can create repositories and push code as you. Keep it in `.env`, keep `.env` in `.gitignore`, give it the shortest expiry you can live with, and grant only the permissions the job needs. If one ever leaks, revoke it on GitHub immediately. A fine-grained token limited to one repository is much safer than a classic token with full access.

19 The GitHub client

```

client = MultiServerMCPClient({
    'github': {
        'transport': 'streamable_http',
        'url': 'https://api.githubcopilot.com/mcp/',
        'headers': {
            'Authorization': f"Bearer {os.environ['GITHUB_PERSONAL_ACCESS_TOKEN']}"
        },
    },
})

tools = await client.get_tools()
tools_by_name = {tool.name: tool for tool in tools}

```

Two familiar lines. The second one is the dictionary you have now written in four sessions. Printing the tools shows a long list, around fifty of them, covering repositories, issues, pull requests and users.

A small helper wraps each call:

```

async def call(tool, args):
    result = await tool.ainvoke(args)
    # MCP tools return a list of content blocks, the first one holds the text
    text = result[0]['text']
    try:
        text = json.loads(text)
    except json.JSONDecodeError:
        pass

    print(f"\n--- {tool.name} ---\n{text}")
    if isinstance(text, str) and text.startswith('failed'):
        raise RuntimeError(f"{tool.name}: {text}") # stop at the first failed step
    return text

```

- `result[0]['text']` again. GitHub's server is no different from yours.
- GitHub returns JSON inside that text, so `json.loads` turns it into a dictionary you can read fields from. If it is not JSON, `pass` leaves it as text.
- The `raise` stops the script at the first failed step. Without it, a failed repository creation would be followed by a branch creation that cannot work, and so on down the chain.

20 The full pull request flow

Each step is one tool call, and together they are a piece of work a developer does by hand every day.

```
get_me                -> who owns this token (login)
|
create_repository     -> live-mcp-demo, private, auto initialised
|
create_branch         -> live-mcp, from main
|
create_or_update_file -> live_mcp.txt on that branch
|                      (this is also the commit and push)
create_pull_request   -> live-mcp into main
|
update_pull_request   -> new title and body
|
pull_request_review_write -> start a pending review
|
add_comment_to_pending_review -> a comment on line 3 of the file
```

```

me = await call(tools_by_name['get_me'], {})
owner = me['login']

repo = 'live-mcp-demo'
target = {'owner': owner, 'repo': repo}

await call(tools_by_name['create_repository'], {
    'name': repo,
    'description': 'Created through the GitHub MCP server',
    'private': True,
    'autoInit': True,
})

await call(tools_by_name['create_branch'], {
    **target, 'branch': BRANCH, 'from_branch': BASE_BRANCH})

created = await call(tools_by_name['create_or_update_file'], {
    **target,
    'branch': BRANCH,
    'path': FILE_PATH,
    'content': '# Live MCP\n\nThis file was created through the GitHub MCP server.\n',
    'message': 'Add live_mcp.txt',
})

```

- `get_me` needs no arguments and tells you the login of whoever owns the token. Use it instead of typing your username, and the script works for anyone who runs it.
- `{**target, ...}` unpacks owner and repo into every call, so they are written once.
- A question from the room: where is the push? There is no separate push tool. `create_or_update_file` commits and pushes in one step, which is what the GitHub API does underneath.
- The comment needs a pending review to attach to, so `pull_request_review_write` with `method: 'create'` runs before `add_comment_to_pending_review`.

THE POINT OF THE DEMO

As the instructor put it, GitHub is not the magic here. The magic is that after two sessions the code on the screen was readable to everyone: a client, a header, a tool list, a dictionary by name, and calls with arguments. Any other MCP server works the same way. In MCP, for every action there is a tool. You find its name, you pass what it asks for.

21 What went wrong live

What appeared	Cause	Fix
No output at all from a script	<code>asyncio.run(main())</code> missing	Add it. Happened twice today
Couldn't find a server with name testing	The session name did not match the key in the config	Use the same name in both places
failed to create repository ... name already exists	The demo had been run before	Delete the repository and run again. There is no delete tool on the GitHub server, so it is a manual step
no pending review found	A comment was added before a review existed	Call <code>pull_request_review_write</code> first
Scenario 3 gave 401 instead of a scope denial	The token expired during <code>time.sleep(5)</code>	Remove the sleep or raise the token lifetime (section 15)

22 Clarifications from the Q&A

- **Can a stdio server call a stdio server?** Yes. All four combinations work. Use `PythonStdioTransport` for a stdio target and a URL for an HTTP one.
- **Without an LLM, how does the client know which tool to call?** You name it. The model is only needed when something else should choose.
- **Where do client ids and secrets live in production?** In a database, and encoded or hashed rather than in plain text. The dictionary in the demo is only to keep the code short.
- **Is the scope the same as authorization?** Yes. Authentication proves who the caller is, scopes decide what that caller may do.
- **Is `/token` a standard path?** The pattern is standard rather than the exact spelling. Every server publishes a token endpoint and an MCP endpoint, and you read their addresses from its documentation.
- **How is the token validated on each call?** The client puts it in the `Authorization` header, and `JWTVerifier` checks the signature, issuer, audience and expiry before the tool runs.
- **Can we refresh an expired token?** Yes, exactly as with any API. Ask for a new one, or use a refresh token if the server issues them.
- **What if a server has no tool for what you need?** Then you cannot do it through that server. GitHub's MCP server has no tool for deleting a repository, which is why every rerun in class needed a manual delete.
- **Can MCP tools use agents or RAG inside them?** Yes. A tool is ordinary code. It can call a model, a vector store, or another MCP server.

- **The model does not always call the same tools. How can anyone rely on this?** A fair worry, and the answer was honest: models do hallucinate, and a better model helps but does not remove the risk. What removes it is engineering. Strict system prompts, guard rails, a human in the loop for anything destructive, logging and tracing with LangSmith, and checking the real state afterwards instead of trusting the model when it says the order was placed.
- **Will there be assignments?** Yes, informally: connect to another public MCP server, such as Google Drive, and try the same flow you saw with GitHub.

WHERE THE COURSE GOES NEXT

MCP is finished for now. The next topic is RAG, and MCP will come back when the class builds full projects, where tools, agents and RAG are combined.

23 Key takeaways and official links

- **A server can be a client.** Put a FastMCP `Client` inside a tool and call any other server, in any transport combination.
- **Wrapping another server is a real pattern**, for adding your own checks or cleaning up somebody else's response.
- **Three ways to call a tool:** `ainvoke` with content blocks, `session.call_tool` with `isError`, and FastMCP's `call_tool` with `.data`.
- **Expected errors are `ToolError`**, and their message always reaches the client.
- **Unexpected errors should be masked**, so paths and tracebacks stay on the server.
- **A failed tool is not an exception.** It is a result with `isError` set.
- **Authentication is the ordinary client credentials flow:** id and secret for a token, token in the `Authorization` header.
- **Scopes are authorization**, checked per tool with `require_scopes`, and tools you lack the scope for are hidden entirely.
- **Tokens expire**, and a client has to handle that.
- **Secrets belong in `.env`**, credentials in a database, and tokens get the least permission and shortest life that works.
- **Using a public MCP server is the same code you already wrote**, plus one header.
- **For every action there is a tool.** Read the server's documentation, find the name, pass the arguments.

Official links

What	Link
FastMCP documentation	gofastmcp.com
FastMCP client	gofastmcp.com/clients/client
Tools, errors and annotations	gofastmcp.com/servers/tools
Server configuration and custom routes	gofastmcp.com/servers/server
JWT token verification	gofastmcp.com/servers/auth/token-verification
GitHub MCP server	github.com/github/github-mcp-server
GitHub fine-grained tokens	github.com/settings/tokens
MCP specification	modelcontextprotocol.io/specification/latest

PRACTICE FOR THE WEEK

Three things, in order of effort. Take the error handling demo and add a tool that fails in a new way, then watch what the client sees with masking on and off. Take the auth server and add a third client with a scope of your own, such as `report:read`, and a tool that requires it. Then pick a public MCP server you have not used, Google Drive for example, create a token with the smallest permissions that work, and run a small flow end to end. When something comes back as an unknown tool, remember to check your scopes first.